

APPLICATION DESIGN DOCUMENT

Day to day stock price predictor

Group	15
Team members	Dekker, Lucan den (251285), Gádor, Gergely (250713), Kambarov, Timur (250596), Qusaily, Qusai Al (236866)
Mentor	Rajendran, Elavendan
Version	3.0
Date	16/05/2026

Revision History

Record each meaningful change to this document. Update the version number (e.g. 0.1 → 0.2 → 1.0) whenever you share a new copy with your team or coach.

Version	Date	Author(s)	Summary of changes
1.0	16/05/2026	Dekker, Lucan den (251285), Gádor, Gergely (250713), Kambarov, Timur (250596), Qusaily, Qusai Al (236866)	Initial version of the application design document. Sections 1 (Introduction), 2 (System Architecture), 3 (Application Components), and 4 (Data Flow) completed in full. Section 5 (Data Storage and Database Design) left pending, awaiting completion by the responsible team member. Section 6 (Design Decisions and Trade-offs) left pending, as the dashboard wireframes had not yet been finalised.
2.0	19/05/2026	Dekker, Lucan den (251285), Gádor, Gergely (250713), Kambarov, Timur (250596), Qusaily, Qusai Al (236866)	Completed Section 5 (storage approach, ERD diagram, entity tables, relationships, data lifecycle) and Section 6 (key design decisions table, all quality attribute paragraphs). Corrected several inaccuracies throughout the document: task framing updated from regression to classification, evaluation metrics corrected from RMSE/MAPE to Accuracy/F1, model references updated, and train/test split corrected from 70/15/15 to 80/20. Updated preprocess.py with schema validation and Pydantic checks.
3.0	22/05/2026	Dekker, Lucan den (251285), Gádor, Gergely (250713), Kambarov, Timur (250596), Qusaily, Qusai Al (236866)	Completed Section 6 (Design Decisions and Trade-offs). Added key design decisions table covering model architecture, dashboard framework, database technology, and prediction target. Completed all quality attribute paragraphs (usability, reliability, maintainability, scalability, privacy and security). Updated Section 7 to reflect that Section 6 is now resolved.

Table of Contents

Revision History	2
Table of Contents	3
1. Introduction	4
1.1 Purpose of this Document	4
1.2 Project Context	4
1.3 Scope	4
2. System Architecture	4
2.1 Architecture Overview	4
2.2 System Architecture Diagram	5
2.3 Technology Stack	6
3. Application Components	7
3.1 Component Overview	7
3.2 Component Descriptions	7
3.2.1 Data Ingestion Module	7
3.2.2 Preprocessing Pipeline	7
3.2.3 Machine Learning Module	8
3.2.4 Database Layer	8
3.2.5 Streamlit Dashboard	8
3.3 Component Interactions	9
4. Data Flow	10
4.1 Data Sources	10
4.2 Data Flow Diagram	10
4.3 Data Processing Steps	11
4.4 Data Quality and Validation	11
5. Data Storage and Database Design	12
5.1 Storage Approach	12
5.2 Database / Storage Schema Diagram	12
5.3 Entities and Attributes	12
5.4 Relationships	13
5.5 Data Lifecycle	13
6. Design Decisions and Trade-offs	13
6.1 Key Design Decisions	13
6.2 Quality Attributes	13
6.2.1 Usability	13
6.2.2 Reliability	13
6.2.3 Maintainability	13
6.2.4 Scalability	14
6.2.5 Privacy and Security	14

7. Open Issues and Future Work	14
8. References	15
Appendix A. Additional Diagrams and Notes	16

1. Introduction

1.1 Purpose of this Document

This document describes the design of the Day to Day Stock Price Predictor application, including its system architecture, application components, data flow, and data storage solution. It is intended for the development team, the project mentor, and the assessor to provide insight into the technical decisions made during the design and implementation of the application.

1.2 Project Context

The Day to Day Stock Price Predictor addresses the challenge of short-term stock price movement forecasting for Microsoft Corporation (MSFT). Non-professional retail investors often lack access to data-driven tools that can support their trading decisions. This application aims to fill that gap by combining historical MSFT price data with exogenous market signals, including gold prices, crude oil prices, and the VIX volatility index, to predict whether the next-day closing price will move up or down. The project was commissioned as part of the Block D capstone of the ADS&AI programme at Breda University of Applied Sciences, in response to the Nasdaq-100 Data Challenge brief.

1.3 Scope

In scope:

- Next-day binary directional prediction (up/down) for MSFT closing price
- Machine learning pipeline using historical price and exogenous features
- Interactive Streamlit dashboard for displaying predictions and visualisations
- SQLite database for data storage and prediction persistence
- Docker containerisation for portability

Out of scope:

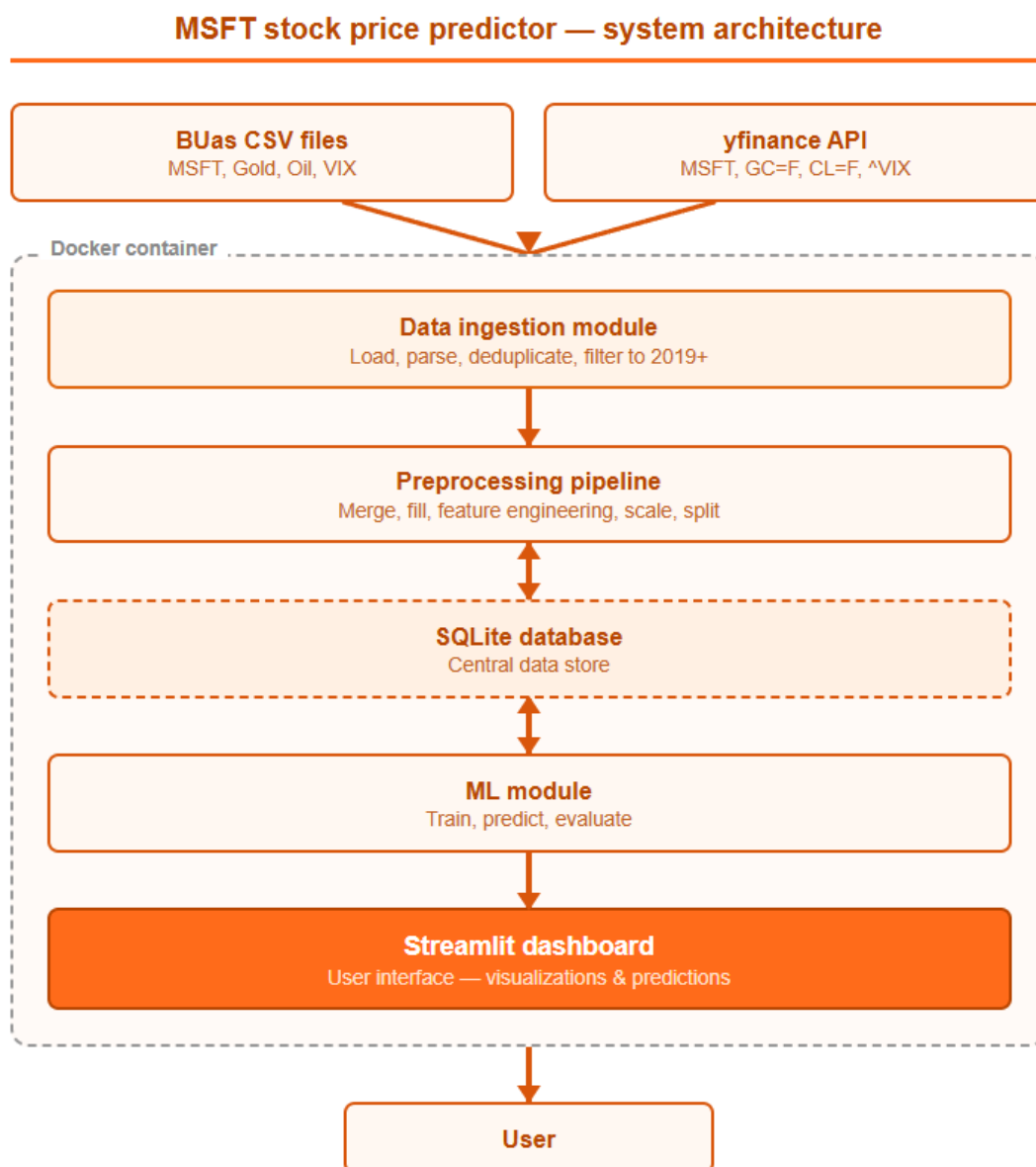
- Real-time or live market data streaming
- Mobile application
- User authentication or personalised user accounts
- Trading execution or direct broker integration
- Predictions for stocks other than MSFT
- Intraday (minute-level) price predictions

2. System Architecture

2.1 Architecture Overview

The Day to Day Stock Price Predictor follows a layered pipeline architecture consisting of four main layers: data ingestion, data processing and storage, machine learning, and presentation. Raw financial data is loaded from CSV files into a SQLite database, which serves as the central data store for the entire project. A preprocessing pipeline reads from this database, engineers features, and writes processed data back to it. The machine learning models are trained on these processed features and write their predictions to the database. The Streamlit dashboard reads directly from the database to present predictions and visualisations to the end user. This architecture was chosen because it keeps all components loosely coupled, ensures the dashboard always displays the latest results, and makes the full pipeline reproducible by any team member with a single Docker container.

2.2 System Architecture Diagram



2.3 Technology Stack

Layer / Concern	Technology	Why this choice
Data storage	SQLite	Lightweight, file-based, no external server required
Data processing	Python, pandas	Industry standard for tabular data manipulation
Feature scaling	scikit-learn RobustScaler	Resistant to extreme values common in financial data
Machine learning	scikit-learn, XGBoost, Keras	Cover linear, tree-based, and deep learning approaches
Dashboard	Streamlit	Combines frontend and backend in Python, no separate server needed
Containerisation	Docker	Single container ensures portability across all team members' machines
Dependency management	Poetry	Reproducible environments via lock file

3. Application Components

3.1 Component Overview

The application consists of five main components: the Data Ingestion Module, which loads raw financial CSV files into the SQLite database; the Preprocessing Pipeline, which reads raw data from the database, engineers features, and writes processed data back; the Machine Learning Module, which trains models on processed features and stores predictions; the Database Layer, which serves as the central data store for the entire project; and the Streamlit Dashboard, which reads from the database and presents predictions and visualisations to the end user.

3.2 Component Descriptions

3.2.1 Data Ingestion Module

- Purpose: loads raw financial CSV files into the SQLite database for use by the rest of the pipeline.
- Inputs: CSV files for MSFT stock data, gold prices, crude oil prices, and the VIX index.
- Outputs: populated database tables (msft_daily, gold_prices, oil_prices, vix_data).
- Key responsibilities:
 - reading and parsing CSV files with varying formats
 - handling overlapping date ranges between old and new data files
 - removing duplicate rows based on date
 - filtering data to the relevant date range (2019 onwards)
 - writing cleaned raw data to the SQLite database
- Dependencies: sqlite3, pandas, logging

3.2.2 Preprocessing Pipeline

- Purpose: transforms raw data from the database into model-ready features and writes them back to the database.
- Inputs: raw tables from the SQLite database (msft_daily, gold_prices, oil_prices, vix_data).
- Outputs: processed_features table in the SQLite database, fitted RobustScaler saved to disk.
- Key responsibilities:
 - merging all datasets on trading date using left joins with MSFT as the base
 - handling missing values via forward fill
 - engineering lag features, rolling averages, daily returns, and price range
 - creating the binary target column using a 0.5% return threshold
 - scaling features using RobustScaler fitted on training data only
 - performing a time-based 80/20 train/test split
- Dependencies: pipeline/preprocess.py, scikit-learn, pandas, sqlite

3.2.3 Machine Learning Module

- Purpose: trains machine learning models on processed features and writes next-day predictions to the database.
- Inputs: processed_features table from the SQLite database.
- Outputs: predictions table in the SQLite database, serialised model files (.pkl).
- Key responsibilities:
 - training Logistic Regression, Random Forest, XGBoost, and LSTM models iteratively
 - evaluating models using accuracy, precision, recall, and F1 score
 - combining the best models into a stacking ensemble
 - writing final predictions to the database
- Dependencies: scikit-learn, XGBoost, Keras, pandas, sqlite3

3.2.4 Database Layer

- Purpose: serves as the central data store shared by all other components.
- Inputs: data written by the ingestion module, pipeline, and ML module.
- Outputs: data read by the pipeline, ML module, and dashboard.
- Key responsibilities:
 - storing raw financial data per source
 - storing processed and scaled features
 - storing model predictions
 - providing shared query functions used across all notebooks and scripts
- Dependencies: SQLite, sqlite3

3.2.5 Streamlit Dashboard

- Purpose: presents model predictions, historical price data, and visualisations to the end user in an interactive interface.
- Inputs: predictions table and processed_features table from the SQLite database.
- Outputs: interactive web interface displaying predictions, charts, and model performance metrics.
- Key responsibilities:
 - reading the latest predictions from the database
 - displaying next-day directional prediction (UP/DOWN) with confidence
 - visualising historical MSFT price data
 - showing model performance metrics
 - providing an accessible interface for non-technical users
- Dependencies: Streamlit, pandas, sqlite3, plotly or matplotlib

3.3 Component Interactions

Scenario 1: running the full pipeline and generating predictions

- Step 1 - the Data Ingestion Module reads the raw CSV files for MSFT, gold, oil, and VIX and loads them into their respective tables in the SQLite database.
- Step 2 - the Preprocessing Pipeline reads the raw tables from the database, merges them on trading date, engineers features, scales them using RobustScaler, and writes the processed features back to the processed_features table.
- Step 3 - the Machine Learning Module reads the processed_features table, trains the models, evaluates performance, and writes the final predictions to the predictions table in the database.
- Step 4 - the fitted scaler is saved to disk so the dashboard can scale any new input data in exactly the same way without refitting.

Scenario 2: a user opens the dashboard

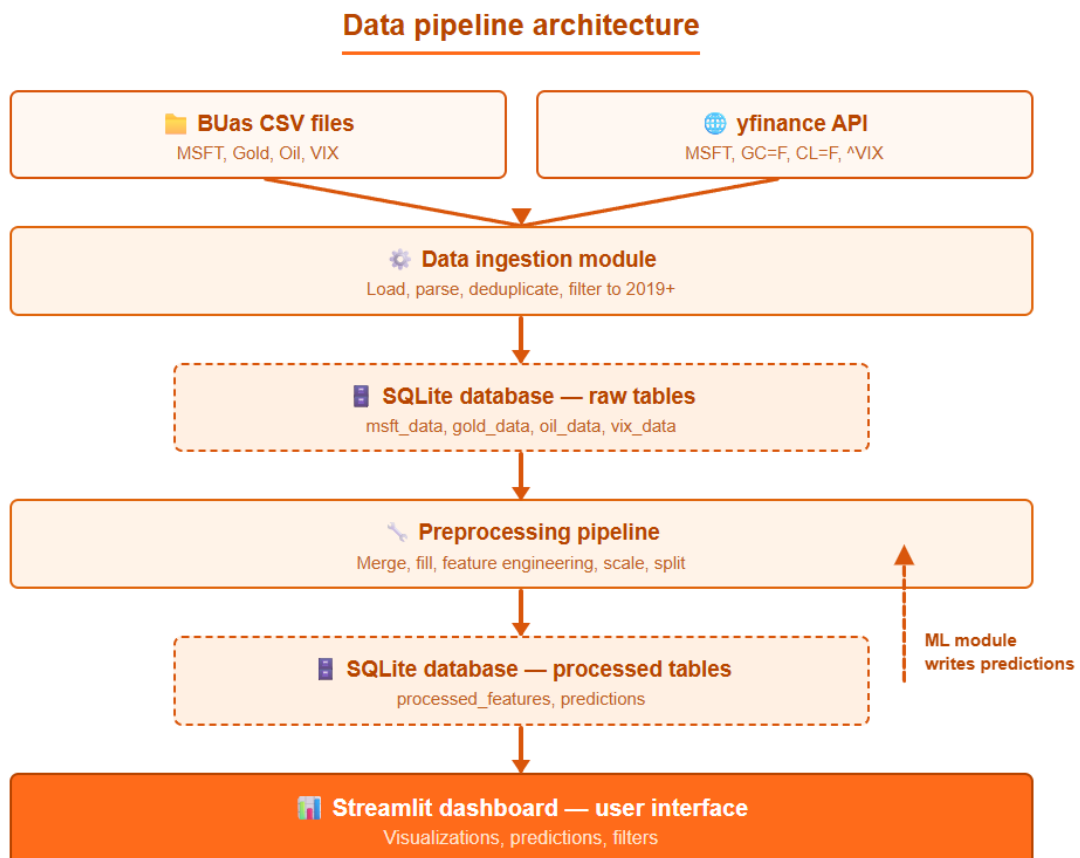
- Step 1 - the user starts the Streamlit application, which establishes a connection to the SQLite database.
- Step 2 - the dashboard reads the latest rows from the predictions table and the processed_features table.
- Step 3 - the dashboard renders the next-day directional prediction (UP or DOWN) along with model confidence, historical price charts, and performance metrics.
- Step 4 - the user interacts with the interface to explore historical predictions and visualisations without any manual intervention required.

4. Data Flow

4.1 Data Sources

Source	Type / Format	Update frequency	Owner
MSFT Daily Stock Data (historical)	CSV, tabular (Date, Open, High, Low, Close, Volume, Transactions)	Historical, static (2015-2025)	BUas
Gold and Crude Oil Prices (historical)	CSV, tabular (Date, Close)	Historical, static (10 years)	BUas
Macroeconomic Indicators VIX (historical)	CSV, tabular (Date, VIX)	Historical, static (10 years)	BUas
MSFT Daily Stock Data (recent)	yfinance API, ticker: MSFT	Daily, up to current date	Yahoo Finance
Gold Prices (recent)	yfinance API, ticker: GC=F	Daily, up to current date	Yahoo Finance
Crude Oil Prices (recent)	yfinance API, ticker: CL=F	Daily, up to current date	Yahoo Finance
VIX Index (recent)	yfinance API, ticker: ^VIX	Daily, up to current date	Yahoo Finance

4.2 Data Flow Diagram



4.3 Data Processing Steps

- **Ingestion:** The data ingestion module loads historical CSV files from BUAs and fetches recent data via the yfinance API, removes duplicates on date, and writes the cleaned raw data into the SQLite database.
- **Merging:** The preprocessing pipeline reads all four raw tables and merges them on trading date using left joins with MSFT as the base index.
- **Cleaning:** Missing values caused by trading calendar misalignments are handled using forward fill. Remaining NaN rows at the start of the dataset are dropped.
- **Feature engineering:** The preprocessing pipeline creates lag features, rolling averages, daily returns, price range, and the binary target column using a 0.5% return threshold.
- **Scaling:** Features are scaled using RobustScaler fitted on training data only. The scaler is saved to disk for reuse by the dashboard.
- **Storage:** Processed features are written back to the SQLite database, from where the ML module reads them for training and the dashboard reads predictions for display.

4.4 Data Quality and Validation

The following checks are applied to ensure incoming data is usable before it enters the pipeline:

- **Duplicate dates:** After combining historical CSV files with recent yfinance data, duplicate rows are removed based on the date column to prevent the same trading day appearing twice in the dataset.
- **Missing values:** After merging the four datasets, each table is checked for missing values. Forward fill is applied to external data columns (gold, oil, VIX) where minor calendar gaps exist. Any rows that still contain NaN after filling are dropped.
- **Row count validation:** After each loading and processing step, the pipeline logs the number of rows loaded and remaining. This allows the team to immediately detect if a file loaded unexpectedly few rows or if too many rows were dropped.
- **Class balance check:** After target creation, the distribution of UP and DOWN labels is logged to confirm the dataset is not severely imbalanced before model training begins.
- **Schema validation:** Column names and data types are verified after loading each CSV to confirm the expected structure is present before merging proceeds.

If any of these checks produce unexpected results, the logging output flags the issue with a warning so it can be identified and corrected before the pipeline continues.

5. Data Storage and Database Design

5.1 Storage Approach

The project uses SQLite as its relational database solution. SQLite is a lightweight, file-based database that requires no external server, no installation beyond Python built-in sqlite3 library, and no configuration. This makes it ideal for a proof-of-concept project of this scale, where the full dataset is under 100MB and only one process writes to the database at a time. The database serves as the central data store for the entire project — raw data is loaded into it, processed features are written back to it, predictions are stored in it, and the dashboard reads from it. Several alternatives were considered. PostgreSQL was evaluated but rejected because it requires a running server and additional configuration, adding unnecessary complexity for a project of this size. DuckDB was also considered as it offers strong analytical query performance, but SQLite was preferred due to its simpler setup and wider familiarity across the team. MongoDB was not considered appropriate because all project data is structured and tabular, making a relational model the natural fit.

5.2 Database / Storage Schema Diagram

The ERD below shows all six tables in the database and their shared date key. All tables are linked through the date column — relationships are established at pipeline runtime through pandas merge operations rather than through database-level foreign key constraints.

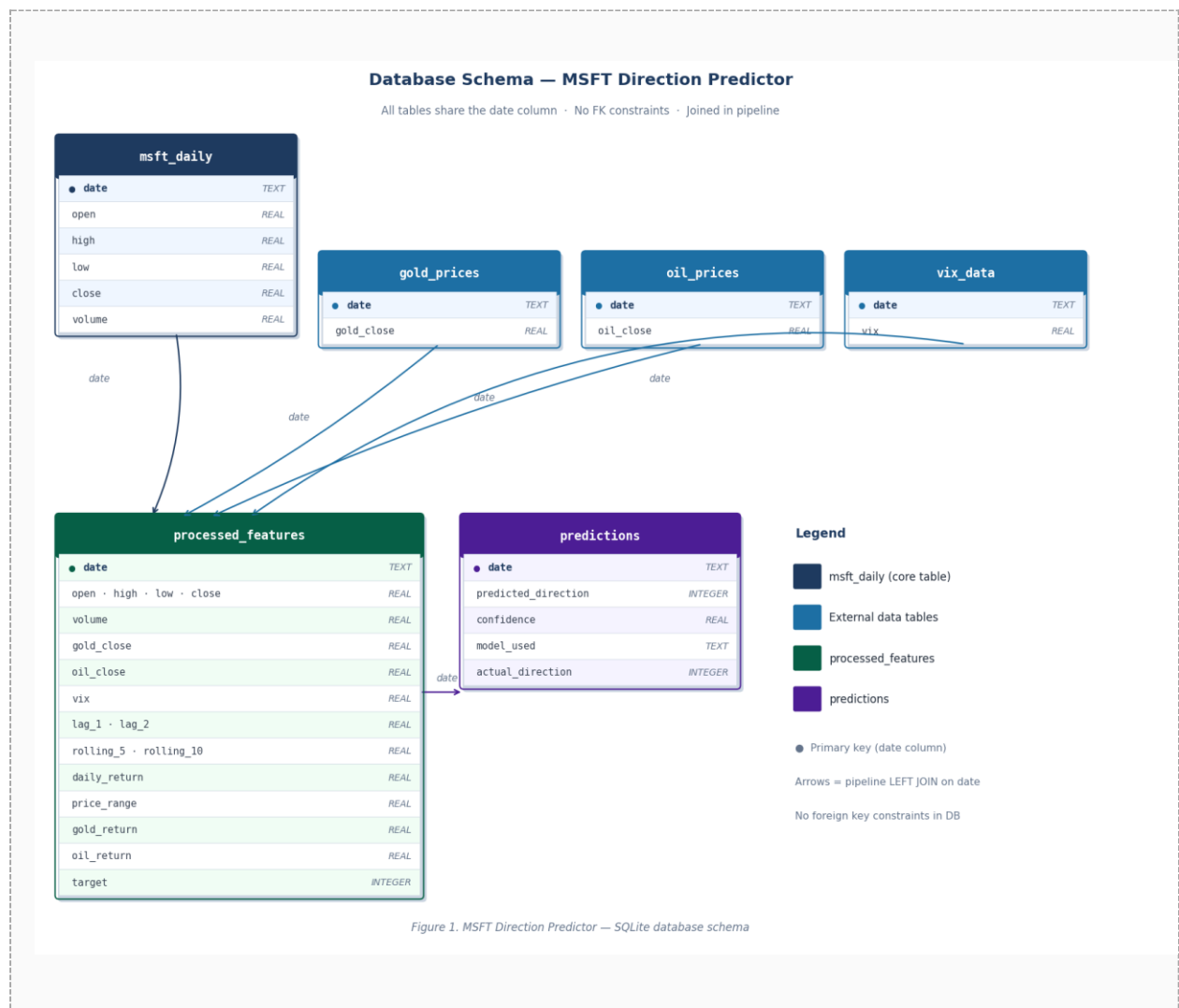


Figure 1. MSFT Direction Predictor database schema. All tables are linked via the date column.

5.3 Entities and Attributes

The database contains six tables. The `msft_daily` table stores raw daily MSFT stock data with columns: `date` (TEXT, PK), `open` (REAL), `high` (REAL), `low` (REAL), `close` (REAL), `volume` (REAL). The `gold_prices` table stores daily gold closing prices with columns: `date` (TEXT, PK), `gold_close` (REAL). The `oil_prices` table stores daily crude oil closing prices with columns: `date` (TEXT, PK), `oil_close` (REAL). The `vix_data` table stores daily VIX volatility index values with columns: `date` (TEXT, PK), `vix` (REAL). The `processed_features` table stores all engineered features ready for model training with columns: `date` (TEXT, PK), `open`, `high`, `low`, `close`, `volume`, `gold_close`, `oil_close`, `vix`, `lag_1`, `lag_2`, `rolling_5`, `rolling_10`, `daily_return`, `price_range`, `gold_return`, `oil_return` (all REAL), `target` (INTEGER, 1=UP 0=DOWN). The `predictions` table stores model output with columns: `date` (TEXT, PK), `predicted_direction` (INTEGER), `confidence` (REAL), `model_used` (TEXT), `actual_direction` (INTEGER).

Attribute	Type	Constraints (PK / FK / NOT NULL / ...)	Description
msft_daily — MSFT daily stock data			
date	TEXT	PK, NOT NULL	Trading date
open	REAL	NOT NULL	Opening price (USD)
high	REAL	NOT NULL	Daily high price (USD)
low	REAL	NOT NULL	Daily low price (USD)
close	REAL	NOT NULL	Closing price (USD)
volume	REAL	NOT NULL	Number of shares traded
gold_prices — Daily gold closing prices			
date	TEXT	PK, NOT NULL	Trading date
gold_close	REAL	NOT NULL	Gold closing price (USD)
oil_prices — Daily crude oil closing prices			
date	TEXT	PK, NOT NULL	Trading date

Attribute	Type	Constraints (PK / FK / NOT NULL / ...)	Description
oil_close	REAL	NOT NULL	Crude oil closing price (USD)
vix_data — Daily VIX volatility index			
date	TEXT	PK, NOT NULL	Trading date
vix	REAL	NOT NULL	VIX market volatility index value
processed_features — Engineered features for model training			
date	TEXT	PK, NOT NULL	Trading date
open	REAL	NOT NULL	MSFT opening price
high	REAL	NOT NULL	MSFT daily high price
low	REAL	NOT NULL	MSFT daily low price
close	REAL	NOT NULL	MSFT closing price
volume	REAL	NOT NULL	MSFT shares traded
gold_close	REAL	NOT NULL	Gold closing price
oil_close	REAL	NOT NULL	Crude oil closing price
vix	REAL	NOT NULL	VIX volatility index
lag_1	REAL	NOT NULL	Previous day closing price
lag_2	REAL	NOT NULL	Two days ago closing price
rolling_5	REAL	NOT NULL	5-day moving average of close
rolling_10	REAL	NOT NULL	10-day moving average of close
daily_return	REAL	NOT NULL	Daily percentage change in close
price_range	REAL	NOT NULL	Daily high minus daily low
gold_return	REAL	NOT NULL	Daily percentage change in gold
oil_return	REAL	NOT NULL	Daily percentage change in oil
target	INTEGER	NOT NULL	1 = UP (>+0.5%), 0 = DOWN (<-0.5%)
predictions — Model output and predictions			
date	TEXT	PK, NOT NULL	Trading date of prediction
predicted_direction	INTEGER	NOT NULL	1 = UP, 0 = DOWN

Attribute	Type	Constraints (PK / FK / NOT NULL / ...)	Description
confidence	REAL	NOT NULL	Model confidence score (0-1)
model_used	TEXT	NOT NULL	Name of model used
actual_direction	INTEGER		Actual outcome (filled after event)

5.4 Relationships

All six tables in the database share the date column as their common key. There are no foreign key constraints defined between tables — relationships are established at pipeline runtime through pandas merge operations rather than at the database level. The preprocessing pipeline performs left joins using MSFT trading dates as the base, bringing in matching rows from gold_prices, oil_prices, and vix_data for each trading day. The resulting merged and processed dataset is written to the processed_features table. The predictions table is populated by the model after training and stores one prediction row per trading date corresponding to rows in processed_features. Each table therefore has a one-to-one relationship with every other table based on the date column.

5.5 Data Lifecycle

The raw data tables (msft_daily, gold_prices, oil_prices, vix_data) are created once when the database management notebook is run. They are fully replaced each time the notebook is rerun using if_exists=replace, which drops and recreates the table with fresh data. The processed_features table is created by the preprocessing pipeline and replaced each time the pipeline runs, ensuring it always reflects the latest raw data and any pipeline changes. The predictions table is created by the model notebook and replaced each time the model runs, always containing the most recent set of predictions. No data is ever partially updated — all writes are full replacements. No personal data is stored in the database at any point. All data is publicly available financial market data sourced from the Nasdaq-100 dataset and the yfinance API. No archiving or deletion policy is required beyond the full replacement approach described above, and GDPR does not apply as no personal data is processed.

6. Design Decisions and Trade-offs

6.1 Key Design Decisions

The table below captures the four most significant design decisions made during the development of the Day to Day Stock Price Predictor. Each decision was arrived at after considering multiple alternatives and evaluating them against the constraints of the project, including development time, team expertise, deployment simplicity, and the educational context of the application.

Decision	Options considered	Choice	Rationale
ML model architecture	Single model (ARIMA, Random Forest, or LSTM); ensemble of multiple models	Stacking ensemble (RF + XGBoost + LSTM, meta-learner: Logistic Regression)	Combining tree-based models with a neural network leverages both linear and non-linear patterns in price data. The stacking meta-learner

Decision	Options considered	Choice	Rationale
			learns which base model to trust under which market conditions, improving generalisation over any single model.
Dashboard framework	Flask + React; Dash; Streamlit	Streamlit	Streamlit integrates natively with Python, scikit-learn, and pandas, removing the need for a separate frontend stack. Its widget model maps directly to the interactive controls in the wireframes (date sliders, model toggles, chart selectors), and the entire team could contribute without frontend expertise.
Database technology	PostgreSQL (server-based relational DB); plain CSV files; SQLite	SQLite	SQLite is serverless and file-based, requiring no separate installation or configuration. It supports full SQL queries and relational structure across multiple tables (raw_data, processed_features, predictions), which plain CSV files cannot provide. PostgreSQL would be appropriate at production scale but adds operational overhead that is not justified for a single-user, locally deployed educational application.
Prediction target (regression vs classification)	Regression: predict the exact next-day closing price; Classification: predict directional movement (UP / DOWN)	Binary classification (UP / DOWN)	Retail investors primarily need to know whether to buy, hold, or sell, not the exact price. Framing the problem as binary classification simplifies the model output, makes confidence scores more interpretable, and aligns with the UP/DOWN indicator shown prominently in the wireframe. Exact price prediction is also considerably harder to evaluate meaningfully with

Decision	Options considered	Choice	Rationale
			RMSE on a non-stationary series.

6.2 Quality Attributes

The following paragraphs describe how each quality attribute was addressed in the design of the Day to Day Stock Price Predictor, and which trade-offs were accepted.

6.2.1 Usability

Usability means that the target user, a non-professional retail investor with limited technical knowledge, can understand the application's output without reading documentation. The wireframe addresses this through several concrete choices. The prediction result is displayed as a large directional indicator (UP or DOWN) with a colour-coded arrow, so the takeaway is immediately visible above the fold. A confidence score accompanies every prediction, helping users judge how much weight to place on the result. The four-page navigation structure (Overview, Historical Analysis, Model Performance, About & Methodology) separates the daily actionable output from deeper exploratory content, so casual users are not overwhelmed. A persistent disclaimer banner on every page ensures users are aware the tool is for educational purposes only. The trade-off accepted is that simplifying the output to a binary direction hides the underlying price forecast detail, which a more advanced user might want.

6.2.2 Reliability

Reliability means the application behaves predictably even when external dependencies fail. The main reliability risk in this project is the yfinance API, which is a third-party service that may be temporarily unavailable. The data ingestion module handles this with try/except blocks that catch network and parsing errors and write structured error messages to the application log, preventing silent failures. If fresh data cannot be fetched, the pipeline falls back to the most recently cached data in the SQLite database so the dashboard can still display the last valid prediction with a visible staleness warning. All pipeline stages log their progress and any exceptions using Python's logging module, which makes post-hoc debugging possible. The trade-off is that the fallback prediction may be up to one trading day old, which is made explicit to the user via the prediction date displayed on the Overview page.

6.2.3 Maintainability

Maintainability means a new team member or assessor can read, understand, and extend the codebase without requiring assistance from the original authors. The project enforces this through several structural rules applied to all code in the repository. Every function and module is documented with a docstring that describes its purpose, parameters, and return values. All non-trivial logic is accompanied by inline comments. Module boundaries map directly to pipeline stages (ingestion, preprocessing, modelling, serving), so a developer can modify one stage without understanding the others. PEP 8 formatting is applied consistently across the codebase, and the repository includes a README that describes how to set up and run the application. The trade-off is that enforcing these standards adds overhead during development, particularly for early exploratory notebooks where strict structure can slow iteration.

6.2.4 Scalability

Scalability in this context means understanding where the current design would break down if usage grew significantly. The most likely bottleneck is SQLite, which does not support

concurrent write access from multiple processes or users. If the application were deployed as a shared web service for many simultaneous users, concurrent dashboard reads and scheduled pipeline writes could produce locking errors. A second bottleneck is the batch retraining process: the current design retrains all models on the full historical dataset once per day, which may become slow if the feature set or dataset grows substantially. The application is not designed for multi-user or cloud deployment, and this is an acknowledged limitation consistent with the educational scope of the project. A future migration to PostgreSQL and asynchronous task scheduling (such as Celery) would address both bottlenecks.

6.2.5 Privacy and Security

The application does not collect, store, or process any personal data from users. All data handled by the system consists of publicly available financial market prices (MSFT, Gold, Crude Oil, VIX) retrieved from Yahoo Finance and the CBOE. Because no personal data is processed, the application falls outside the scope of GDPR data subject rights obligations. The main data protection concern is the integrity of the SQLite database file, which is stored locally on the machine running the application. Access is controlled by the operating system's file permissions. API keys (if required by future data sources) are stored as environment variables and are never committed to the repository. The yfinance library is used for personal and educational purposes only, consistent with its terms of use, and the application includes a disclaimer to that effect on every dashboard page.

7. Open Issues and Future Work

- Known limitation: The yfinance API is intended for personal and educational use only, which limits the application to non-commercial use.
- Known limitation: SQLite is file-based and does not support concurrent access, meaning the application is limited to single-user local deployment.
- Known limitation: The model is not retrained automatically, meaning prediction quality may degrade over time as market conditions change.
- Future improvement: The current architecture is built around MSFT as a single stock. A future improvement would be to generalise the pipeline and dashboard to support multiple Nasdaq-100 stocks, allowing users to select any ticker and receive next-day directional predictions for it.

8. References

Roussi, R. (2023). *yfinance: Download market data from Yahoo! Finance's API*. Retrieved from Github: <https://github.com/ranaroussi/yfinance>

SQLite . (2024). *SQLite Documentation*. Retrieved from <https://www.sqlite.org/docs.html>

Streamlit. (2024). *Streamlit documentation*. Retrieved from <https://docs.streamlit.io>

Appendix A. Additional Diagrams and Notes

Optional. Use this section for anything that supports the design but would clutter the main body: larger diagrams, pseudocode, API samples, detailed configuration, or experimental results that informed a decision.

(SQLite , 2024) (SQLite , 2024) (Streamlit, 2024) (Roussi, 2023)